

Laporan Milestone 4 & 5

Semantic Analysis

IF2224 Teori Bahasa Formal dan Automata



Kelompok JEE - SundalnJVM

Billie Bhaskara Wibawa	13524024
Raysha Erviandika Putra	13524050
Muhammad Naufal Romi Annafi	13524058
Dzaki Ahmad Al Hussainy	13524084

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026

Daftar Isi

Daftar Isi.....	1
1. Dasar Teori.....	2
1.1. Intermediate Code dan Decorated AST.....	2
1.2. Three Address Code.....	2
1.3. Translasi Control Flow.....	3
1.4. Interpreter.....	3
1.5. The Stack.....	3
1.6. Runtime Type Checking.....	4
2. Perancangan dan Implementasi.....	5
2.1. Teknologi yang Digunakan.....	5
2.2. Arsitektur Program.....	5
2.3. Alur Eksekusi Program.....	6
2.4. Kelas Instruction dan RuntimeValue.....	6
2.5. Kelas CodeGenerator.....	8
2.6. Kelas Interpreter.....	10
3. Pengujian.....	12
4. Kesimpulan dan Saran.....	21
5. Lampiran.....	23
6. Referensi.....	27

1. Dasar Teori

1.1. Intermediate Code dan Decorated AST

Intermediate Code adalah representasi struktural dari program komputer yang berlokasi di antara high-level language dan machine code. Tujuan utama dari pembentukan Intermediate Code adalah memfasilitasi modularitas arsitektur, yaitu memisahkan proses front-end yang mencakup analisis leksikal, sintaks, dan semantik dari proses back-end yaitu code generation. Pendekatan ini memungkinkan compiler untuk melakukan machine-independent optimization sebelum machine code dibentuk.

Decorated AST merupakan struktur Abstract Syntax Tree yang setiap node di dalamnya telah ditambahkan atribut informasi semantik. Setelah fase parsing menghasilkan Abstract Syntax Tree, fase semantic analysis akan melakukan traversal pada tree tersebut dan menyematkan atribut teknis seperti:

- Tipe data dari sebuah ekspresi.
- Referensi menuju symbol table.
- Batasan scope dari sebuah variabel.

Decorated AST ini kemudian digunakan sebagai input utama pada proses generation untuk membentuk Intermediate Code.

1.2. Three Address Code

Three-Address Code merupakan salah satu bentuk linear Intermediate Code yang paling ekstensif digunakan. Format ini mendekomposisi ekspresi kompleks dari Abstract Syntax Tree menjadi instruksi linear yang sangat spesifik. Aturan dasar pada arsitektur ini menetapkan bahwa setiap instruksi maksimal memuat tiga operand. Operand tersebut umumnya terdiri dari dua source operand dan satu destination operand.

Untuk mengevaluasi ekspresi yang panjang, compiler akan men-generate temporary variable. Sebagai contoh, instruksi $x = a + b * c$ akan ditranslasikan menjadi bentuk berikut:

- $t1 = b * c$, dengan $t1$ sebagai temporary variable untuk hasil perkalian.
- $t2 = a + t1$ untuk operasi penjumlahan lanjutan.
- $x = t2$ sebagai assignment nilai akhir ke variabel x

1.3. Translasi Control Flow

Translasi control flow merupakan proses konversi struktur kontrol high-level seperti percabangan dan perulangan menjadi jump instruction primitif yang dieksekusi secara sekuensial oleh mesin. Proses ini berfokus pada evaluasi kondisi boolean dan alokasi label pada alamat memori mesin eksekusi. Instruksi tersebut dikategorikan menjadi dua jenis:

- Unconditional Jump: Pemindahan alur eksekusi secara absolut menuju suatu label.
- Conditional Jump: Pemindahan alur eksekusi yang dipicu oleh hasil evaluasi sebuah kondisi boolean.

Pada implementasi blok perulangan, instruksi ditranslasikan dengan menyisipkan conditional jump menuju akhir blok instruksi apabila kondisi bernilai false. Sistem kemudian menyisipkan unconditional jump pada akhir blok iterasi untuk mengembalikan alur eksekusi ke awal pengecekan kondisi.

1.4. Interpreter

Interpreter merupakan program yang mengeksekusi source code atau Intermediate Code secara langsung per instruksi, tanpa melalui proses kompilasi menyeluruh menjadi executable machine code. Berbeda dengan pendekatan ahead-of-time compilation, interpreter mengintegrasikan proses analisis dan eksekusi dalam satu waktu kerja, atau menjalankan instruksi bytecode langsung pada sebuah virtual machine. Meskipun performa runtime cenderung lebih lambat dibandingkan compiled code, pendekatan ini menyediakan tingkat fleksibilitas yang tinggi, siklus debugging yang efisien, serta kemudahan portabilitas lintas platform.

1.5. The Stack

Pada runtime architecture, stack merupakan struktur data untuk alokasi memori dinamis yang mengimplementasikan prinsip last-in-first-out. Komponen ini berfungsi krusial dalam manajemen execution flow, khususnya saat menangani function call.

Setiap kali terjadi instruksi pemanggilan fungsi, sistem akan melakukan operasi push untuk memasukkan blok memori baru berupa activation record atau stack frame ke dalam stack. Struktur data ini menyimpan:

- Argumen yang di-pass menuju fungsi.
- Variabel lokal yang dideklarasikan dalam scope fungsi tersebut.

- Return address untuk menentukan lokasi instruksi eksekusi berikutnya setelah siklus fungsi selesai.

Setelah eksekusi fungsi mengalami terminasi, sistem akan menjalankan operasi pop pada stack frame untuk membebaskan memori dan mengembalikan alur kontrol kepada caller.

1.6. Runtime Type Checking

Runtime type checking merupakan mekanisme sistem untuk memvalidasi tipe data pada variabel atau ekspresi secara dinamis tepat saat instruksi dieksekusi, dan bukan pada fase compile-time. Mekanisme ini merupakan fondasi utama dari bahasa pemrograman yang menerapkan sistem dynamic typing.

Pada sistem tersebut, variabel tidak memiliki alokasi permanen dengan suatu tipe data spesifik. Sebuah variabel dapat menyimpan nilai integer pada satu waktu, lalu menampung tipe string pada instruksi selanjutnya. Sistem interpreter akan memverifikasi kompatibilitas tipe sebelum sebuah instruksi matematis atau pemanggilan method dieksekusi. Apabila terdeteksi ketidaksesuaian operasi dengan tipe data, program akan memicu type error exception seketika pada saat runtime.

2. Perancangan dan Implementasi

2.1. Teknologi yang Digunakan

Program ini dibangun menggunakan bahasa C++ dengan memanfaatkan Standard Template Library (STL) seperti `std::vector`, `std::string`, dan `std::shared_ptr`. Program juga menggunakan `makefile` untuk mengotomatisasi proses kompilasi program. Alasan penggunaan bahasa C++ beserta STL untuk memaksimalkan performa dan menjamin keamanan memori, serta konfigurasi dengan `makefile` untuk memastikan siklus kompilasi yang otomatis, konsisten, dan efisien.

2.2. Arsitektur Program

Secara garis besar, arsitektur program dapat dibagi menjadi bagian seperti Lexical Analyzer, Syntax Analyzer, Semantic Analyzer, Immediate Code Generation, Interpreter.

Immediate Code Generation

Bagian ini diimplementasikan oleh kelas `CodeGenerator` sebagai turunan `ASTVisitor` yang melakukan traversal AST. Kelas ini menerjemahkan AST ke intermediate code dengan pendekatan stack-based dan activation record. Informasi semantic seperti lexical level dan offset variabel diambil dari `SymbolTable`.

State utama `CodeGenerator`:

- `instructions`: Menyimpan hasil instruksi.
- `symbolTable`: Referensi untuk informasi semantic.
- `subroutineAddresses` & `unresolvedCalls`: Digunakan untuk proses backpatching alamat fungsi.

Struktur Instruction:

Setiap instruksi terdiri dari `OpCode` yang mewakili instruksi virtual machine seperti LIT, LOD, STO, CAL, JMP, OPR, RET. Struktur ini juga menyimpan level, operand, dan `RuntimeValue` untuk menangani nilai literal secara generik.

Interpreter

Kelas Interpreter mengeksekusi vector `Instruction` yang dihasilkan dari code generator. Interpreter menggunakan model eksekusi stack based machine sederhana dengan beberapa pointer utama:

- `pc`: Program counter.
- `sp`: Stack pointer.

- bp: Base pointer untuk activation record.

Metode utama run dan execute memproses loop eksekusi instruksi secara berurutan. Kelas ini juga menyertakan fungsi validasi runtime seperti `validateJumpTarget` dan `ensureStackIndex` untuk memastikan keamanan eksekusi program dan mencegah kesalahan akses memori.

Arsitektur ini mengikuti implementasi umum dari compiler umumnya yang dapat dibagi menjadi dua bagian:

- Frontend: Lexer, parser, dan semantic analyzer menghasilkan AST dan SymbolTable.
- Backend: CodeGenerator menerima AST yang sudah divalidasi dan menghasilkan IR berupa instruksi virtual machine dan interpreter menerima vector instruction tersebut dan mengeksekusinya.

2.3. Alur Eksekusi Program

Secara garis besar alur eksekusi dari program ini adalah

Input → Lexical Analyzer → Syntax Analyzer → Semantic Analyzer → Intermediate Code → Interpreter → Output

Input dimulai dari pembacaan teks dari file dengan extension .arion lalu menyerehkannya ke Lexical Analyzer. Lexical Analyzer akan membaca teks program dan menghasilkan token. Setelah itu, Syntax Analyzer akan mengubah kumpulan token menjadi struktur parse tree untuk mengecek kebenaran sintaks program. Selanjutnya, Semantic Analyzer akan memeriksa aturan bahasa di parse tree dari Syntax Analyzer dan mengubahnya menjadi Decorated AST dan sebuah Symbol Table yang berisi informasi level lexical dan alamat variabel atau fungsi. Setelah semantic analysis selesai, CodeGenerator di codegen mengambil AST yang sudah diverifikasi dan menghasilkan intermediate code secara langsung. Intermediate code yang dihasilkan kemudian dieksekusi oleh Interpreter. Interpreter ini adalah virtual machine stack-based sederhana. Interpreter melakukan loop eksekusi dan akhirnya menghasilkan output dari program.

2.4. Kelas Instruction dan RuntimeValue

Kelas Instruction merupakan unit utama intermediate code pada program ini. Meskipun implementasinya berupa struct di `Instruction.hpp`, perannya sama seperti unit di intermediate representation yaitu mewakili satu langkah virtual machine.

Kelas ini tidak persis tiga alamat tradisional ($x = y \text{ op } z$), tetapi tetap berada pada level IR. Instruction lebih mirip bytecode atau linear intermediate code yang menggunakan:

- satu OpCode,
- satu level untuk lexical/static link,
- satu operand untuk offset/jump target/operator,
- dan kadang RuntimeValue untuk literal.

Jadi arsitektur intermediate code-nya adalah bentuk linear IR yang dapat dikatakan three-address-like karena setiap instruksi membawa opcode dan operand yang jelas, meskipun tidak selalu membawa tiga alamat eksplisit.

```
enum class OpCode {
    LIT, LOD, STO, CAL, INT, JMP, JPC, OPR, RET, LDA, LDI, STI
};

struct Instruction {
    int line;
    OpCode op;
    int level;
    int operand;
    bool hasLiteral;
    RuntimeValue literal;

    Instruction(OpCode op, int level, int operand);
    Instruction(OpCode op, int level, RuntimeValue literal);
    Instruction(int line, OpCode op, int level, int operand);
    Instruction(int line, OpCode op, int level, RuntimeValue literal);
};
```

Penjelasan komponen:

- line: nomor baris sumber untuk pelaporan runtime/error.
- op: opcode virtual machine yang menjelaskan operasi.
- level: lexical depth untuk mengakses frame aktivasi yang benar.
- operand: nilai umum, bisa berupa offset variabel, alamat jump, jumlah ruang stack, atau kode operator.
- hasLiteral: flag apakah instruksi membawa literal langsung.
- literal: nilai literal aktual jika hasLiteral == true.

Dalam implementasinya di CodeGenerator, instruksi LIT sering dibuat dengan RuntimeValue langsung, sedangkan instruksi lain menggunakan operand. Misalnya:

- LIT level literal untuk push nilai konstanta,
- LOD level operand untuk load variabel dari frame,
- STO level operand untuk store variabel,

- CAL level operand untuk panggil subprogram,
- JMP operand / JPC operand untuk kontrol alur.

Sedangkan, RuntimeValue adalah tipe generic untuk semua nilai yang dapat muncul pada saat runtime. Keals ini digunakan terutama di Instruction untuk literal LIT, stack interpreter untuk menyimpan nilai sementara, serta operasi runtime seperti OPR dan I/O.

```
using RuntimeValue = std::variant<int, double, bool, char, std::string>;
```

Tipe ini mengizinkan interpreter dan code generator menggunakan satu representasi nilai tanpa membuat hierarki kelas khusus untuk setiap tipe.

Penjelasan komponen:

- int: bilangan bulat
- double: bilangan pecahan
- bool: nilai logika
- char: karakter tunggal
- std::string: teks / string

2.5. Kelas CodeGenerator

Kelas CodeGenerator merupakan komponen utama untuk menerjemahkan AST yang sudah divalidasi oleh semantic analyzer menjadi intermediate code berupa vector Instruction. Kelas ini mewarisi ASTVisitor sehingga mekanismenya adalah visitor yaitu setiap node AST dikunjungi, lalu CodeGenerator mengeluarkan instruksi yang sesuai.

```
class CodeGenerator : public ASTVisitor {
private:
    std::vector<Instruction> instructions;
    const SymbolTable* symbolTable;
    std::unordered_map<int, int> subroutineAddresses;
    std::vector<std::pair<int, int>> unresolvedCalls;

public:
    CodeGenerator(const SymbolTable* symTab);
    void generate(ProgramNode* root);
    const std::vector<Instruction>& getInstructions() const;

    void visit(ProgramNode* node) override;
    void visit(ProcedureDeclarationNode* node) override;
    void visit(FunctionDeclarationNode* node) override;
    void visit(AssignmentStatementNode* node) override;
    void visit(ProcedureCallNode* node) override;
    // ... banyak visit lainnya ...
};
```

Kelas ini memiliki atribut sebagai berikut:

- `std::vector<Instruction> instructions`

Tempat menampung intermediate code yang dihasilkan.

- `const SymbolTable* symbolTable`

Referensi ke symbol table hasil semantic analysis yang digunakan untuk mengisi level, operand, adr, dan ref saat menerjemahkan variabel dan subprogram.

- `std::unordered_map<int, int> subroutineAddresses`

Peta dari indeks tabel simbol subprogram ke alamat instruksi awal subprogram yang dipakai untuk memperbaiki target panggilan CAL setelah alamat fungsi/prosedur diketahui.

- `std::vector<std::pair<int, int>> unresolvedCalls;`

Daftar CAL yang belum punya target operand saat pertama kali diemit yang setiap entry menyimpan indeks instruksi dan indeks tabel simbol tujuan.

Kelas ini memiliki method sebagai berikut:

- `generate(ProgramNode* root)` untuk membersihkan state dan melakukan traversal AST lewat `root->accept(this)`
- `resolveFunctionCalls()` untuk menyelesaikan operand CAL yang ditunda
- `emit(OpCode op, int level, int operand)` dan `emitLiteral(int level, RuntimeValue value)` adalah helper untuk membuat instruksi.
- `backpatch(int instructionIndex, int targetAddress)` memperbaiki alamat jump atau call setelah tujuan diketahui.

Alur kerja kelas ini dimulai dari node `ProgramNode`, lalu `CodeGenerator` mengemit INT untuk area global, lalu mengunjungi deklarasi dan statement. Saat menemukan deklarasi prosedur/fungsi, ia menulis JMP sementara untuk melewati tubuh subprogram, mencatat alamat awal, lalu mengemit INT, melakukan traversal body, dan menutup dengan RET. Untuk statement seperti if atau while, ia menggunakan JPC/JMP plus backpatch. Untuk `ProcedureCallNode` dan `FunctionCallNode`, argumen digenerasi dulu, kemudian CAL diemit dengan operand placeholder dan dicatat di `unresolvedCalls`. Di akhir, `resolveFunctionCalls()` mengisi operand tersebut menggunakan `subroutineAddresses`.

2.6. Kelas Interpreter

Kelas Interpreter adalah mesin eksekusi untuk intermediate code. Kelas ini mengimplementasikan mesin stack-based sederhana dengan activation record untuk memanggil subprogram. Kelas ini memiliki fungsi untuk Menjalankan vektor Instruction yang dihasilkan CodeGenerator, menyediakan stack runtime bertipe RuntimeValue, serta menangani pemanggilan fungsi/prosedur, kontrol alur, operasi aritmatika/IO, dan pemeriksaan runtime.

```
class Interpreter {
public:
    static constexpr int STACK_MAX = 2048;
    Interpreter();
    explicit Interpreter(std::vector<Instruction> instructions);
    void setInstructions(std::vector<Instruction> instructions);
    void run(std::ostream& out);
    const std::string& getOutput() const;
private:
    std::vector<Instruction> instructions;
    std::vector<RuntimeValue> stack;
    int sp;    // stack pointer
    int bp;    // base pointer (activation record base)
    int pc;    // program counter
    bool halted;
    std::string output;
    // helper methods: reset, execute, executeOpr, base,
    validateJumpTarget, ensureStackIndex, requireStackItems, pop, push,
    toInt, toDouble, isNumeric, isDoubleValue, isEqual, compare
};
```

Penjelasan atribut:

- instructions: vector Instruction yang dieksekusi.
- stack: fixed-size stack (STACK_MAX) menyimpan RuntimeValue untuk nilai lokal, parameter, dan return address.
- sp (stack pointer): indeks top-of-stack.
- bp (base pointer): awal activation record saat ini (static link disimpan di stack[bp]).
- pc (program counter): alamat instruksi berikutnya.
- halted/output: status mesin dan akumulasi keluaran teks.

Penjelasan method:

- run(out): reset state lalu loop: fetch instruksi pada pc, pc++, panggil execute(instr, out) sampai halted.
- execute(instr): dispatch berdasarkan instr.op untuk INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET.
- executeOpr(code, ...): implementasi operator (NEG, ADD, SUB, MUL, DIV, MOD, EQL, comparators, WRT/WRTLN, READLN).

- `base(level)`: menelusuri static links untuk mencetak frame lexical yang sesuai.
- `push/pop` dan helper `toInt/toDouble` mengelola coercion antara varian `RuntimeValue` dan memeriksa error runtime

Alur kerja utama kelas ini adalah Saat run dipanggil, Interpreter men-reset stack dan pointer lalu mulai loop:

- Ambil instruksi di `pc`
- Increase nilai `pc`
- Lalu, jalankan `execute`.

Untuk pemanggilan fungsi, CAL menulis static link (`base(level)`), dynamic link (`bp`), dan return address (`pc`) ke stack, men-set `bp = sp+1` lalu `pc` ke alamat subprogram. Subprogram biasanya memanggil INT untuk mengalokasi ruang. RET me-restore `pc/bp` dari activation record dan menghentikan mesin bila `pc == 0`. Operasi aritmatika/`pop`/`push` dikerjakan lewat `RuntimeValue` sehingga tipe numeric dan string diperlakukan sesuai aturan coercion.

3. Pengujian

3.1. Kasus 1

Source code Arion:

```
program test10;
var
  x, y: Integer;
begin
  x := 10;
  if x > 5 then
    y := x * 2
  else
    y := x div 2;
  writeln(y);
  while y > 0 do
  begin
    y := y - 1;
    if y > 0 then
      writeln(y);
    end;
  end;
end.
```

Hasil byte code:

```
0 INT 0 11
1 LIT 0 10
2 STO 0 3
3 LOD 0 3
4 LIT 0 5
5 OPR 0 11
6 JPC 0 12
7 LOD 0 3
8 LIT 0 2
9 OPR 0 4
10 STO 0 7
11 JMP 0 16
12 LOD 0 3
13 LIT 0 2
14 OPR 0 5
15 STO 0 7
16 LOD 0 7
17 OPR 0 14
18 LOD 0 7
19 LIT 0 0
20 OPR 0 11
21 JPC 0 33
22 LOD 0 7
23 LIT 0 1
24 OPR 0 3
25 STO 0 7
26 LOD 0 7
27 LIT 0 0
28 OPR 0 11
```

```
29 JPC 0 32
30 LOD 0 7
31 OPR 0 14
32 JMP 0 18
33 RET 0 0
```

Output program:

```
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
```

Penjelasan:

“program test10; var x, y: Integer; begin” menghasilkan

INT 0 11

Yang artinya stack pointer ditambahkan 11 untuk memberi space yang cukup bagi program. 11 adalah hasil dari 3 (frame) + 8 (dari x dan y).

“x := 10;” menghasilkan

LIT 0 10

STO 0 3

Yang artinya melakukan push literal 10 ke atas stack, dan kemudian menyimpan 10 yang ada di puncak stack, ke lexical level 0 dan address 3.

“if x > 5 then” menghasilkan

LOD 0 3

LIT 0 5

OPR 0 11

JPC 0 12

Yang artinya melakukan load dari address 3 (variable x tadi) ke atas stack, dilanjut me-load literal 5 ke atas stack, dan dilakukan operasi perbandingan kedua value di atas stack (5 dan 3). Operasi ini direpresentasikan OPR 0 11 karena 11 merupakan mapping untuk operator '>'. Terakhir dilakukan jump ke line 12 apabila operasi tersebut menghasilkan false.

"y := x * 2" menghasilkan

LOD 0 3

LIT 0 2

OPR 0 4

STO 0 7

JMP 0 16

Yang artinya melakukan load dari address 3, dan meload literal 2 ke atas stack. Kemudian dilakukan operasi 4 (mapping untuk *) dan menyimpan hasilnya pada address 7. Setelah itu dilakukan jump ke address 16.

"else y := x div 2;" menghasilkan

LOD 0 3

LIT 0 2

OPR 0 5

STO 0 7

Yang artinya melakukan load dari address 3 dan meload literal 2 ke atas stack. Kemudian dilakukan operasi 4 (mapping untuk div) dan menyimpan hasilnya ke address 7. Tapi kali ini tidak dilakukan jump.

"writeln(y);" menghasilkan

LOD 0 7

OPR 0 14

Yang artinya melakukan load dari address 7 (tempat variable y berada) dan dilakukan operasi 14 yang artinya operasi writeln.

"while y > 0 do" menghasilkan

LOD 0 7

LIT 0 2

OPR 0 11

JPC 0 33

Yang artinya meload variable y dan literal 2 ke atas stack. Dilakukan operasi 11 (>) dan lompat ke address 33 apabila sudah tidak valid (akhir dari loop).

"begin y := y - 1;" merupakan badan loop, menghasilkan:

LOD 0 7

LIT 0 1

OPR 0 3

STO 0 7

Yang artinya meload variable y dan literal 1 ke atas stack, dilakukan operasi 3 (-) dan menyimpan hasilnya ke address tempat y lagi.

"if y > 0 then writeln(y);" menghasilkan

LOD 0 7

LOD 0 0

OPR 0 11

JPC 0 32

LOD 0 7

OPR 0 14

Yang artinya meload variable y, literal 0, dan membandingkannya menggunakan operasi 11 (>) dan lompat ke line 32 apabila hasil operasinya false. Apabila true, maka variabel y dipush lagi ke atas stack dan dilakukan operasi 14 (writeln).

3.2. Kasus 2

Source code Arion:

```
program test11;  
var
```



```

    userInput: String;
begin
    write('Please enter some text: ');

    readln(userInput);
    write('You entered: ');
    writeln(userInput);
end.

```

Hasil byte code:

```

0 INT 0 259
1 LIT 0 "Please enter some text: "
2 OPR 0 13
3 OPR 0 15
4 STO 0 3
5 LIT 0 "You entered: "
6 OPR 0 13
7 LOD 0 3
8 OPR 0 14
9 RET 0 0

```

Output program:

```

Please enter some text: test input
You entered: test input

```

Penjelasan:

"program test11; var userInput: String; begin" menghasilkan

INT 0 259

yang artinya memberi space 259 untuk stack. 259 adalah jumlah dari 3 (frame) + 256 (length default dari string).

"write ('Please enter some text: ');" menghasilkan

LIT 0 "Please enter some text: "

OPR 0 13

Yang artinya meload string "Please enter some text: " ke dalam puncak stack, dan melakukan operasi 13 (write). Kemudian dilakukan operasi 13 (write).

"readln(userInput);" menghasilkan

OPR 0 15

STO 3

Yang artinya melakukan operasi 15 (readln) dan menyimpannya pada address 3 (address tempat variable userInput).

"write('You entered: '); writeln(userInput);" menghasilkan

LIT 0 "You entered: "

OPR 0 13

LOD 0 3

OPR 0 14

Yang artinya meload string "You entered: " ke stack dilanjut operasi 13 (write). Kemudian meload variable pada address 3 dan dilakukan operasi 14 (writeln).

3.3. Kasus 3

Source code Arion:

```
program test1;  
  
var  
  x: Integer;  
  
procedure SetLocal();  
var  
  x: Integer;  
begin  
  x := 10;  
  if x > 5 then  
    x := x + 1;  
end;  
  
begin  
  x := 1;  
  SetLocal();  
end.
```

Hasil byte code:

```
0 INT 0 7  
1 JMP 0 14  
2 INT 0 7  
3 LIT 0 10  
4 STO 0 3  
5 LOD 0 3  
6 LIT 0 5  
7 OPR 0 11  
8 JPC 0 13  
9 LOD 0 3  
10 LIT 0 1  
11 OPR 0 2  
12 STO 0 3  
13 RET 0 0  
14 LIT 0 1  
15 STO 0 3  
16 CAL 0 2
```

```
17 RET 0 0
```

Penjelasan:

“program test1; var x: Integer; procedure SetLocal();” menghasilkan

INT 0 7

JMP 0 14

Yang artinya pertama dilakukan penyiapan space sebanyak 7 (3 (frame) + 4 (variable x)). Dan lompat ke line 14. Hal ini dilakukan untuk melompati definisi procedure agar procedure tidak dijalankan waktu masih didefinisikan.

“begin x := 1; SetLocal();” menghasilkan

LIT 0 1

STO 0 3

CAL 0 2

Yang artinya meload literal 1 dan menyimpannya dalam address 3 (tempat variable x). Kemudian memanggil procedure yang didefinisikan dalam address 2 (tempat didefinisikannya SetLocal()).

“procedure SetLocal(); var x: Integer; begin x := 10;” Ini adalah badan procedure yang didefinisikan. Menghasilkan

INT 0 7

LIT 0 10

STO 0 3

Yang artinya menyiapkan space 7 (3 dari frame + 4 untuk local integer x). Kemudian meload 10 dan menyimpannya dalam address 3. Perlu dicatat bahwa ini address 3 dari lexical level 0, artinya ini address 3 lokal dalam fungsi ini dan tidak mengoverwrite 3 dalam variable global dari program utama.

3.4. Kasus 4

Source code Arion:

```
program test4;  
  
type  
  Person == record
```

```

    nama: String;
    umur: Integer;
end;

var
    p: Person;

begin
    p.nama := 'Budi';
    p.umur := 18;
End.

```

Hasil byte code:

```

0 INT 0 263
1 LDA 0 3
2 LIT 0 0
3 OPR 0 2
4 LIT 0 "Budi"
5 STI 0 0
6 LDA 0 3
7 LIT 0 256
8 OPR 0 2
9 LIT 0 18
10 STI 0 0
11 RET 0 0

```

Penjelasan:

"program test4;

type Person == record nama: String; umur: Integer; end;

var p: Person;

begin

" menghasilkan

INT 0 263

Yang artinya disiapkan space 263 pada stack. 263 hasil dari 3 (frame) + 256 (nama berupa string) + 4 (umur berupa integer).

"p.nama := 'Budi'" menghasilkan

LDA 0 3

LIT 0 0

OPR 0 2

LIT 0 "Budi"

STI 0 0

Yang artinya melakukan push variable pada address 3 ke stack (variable p). Dilanjut dengan melakukan load literal 0 ke stack. Literal ini merepresentasikan memory offset dari nama pada Person. Karena nama adalah atribut pertama, maka offsetnya 0. Kemudian "OPR 0 2" melakukan pop dari stack (yang dipop adalah address 3 yang tadi disimpan) dan dilakukan operasi 2 (+) untuk literal 0 dan address 3 dan mendorong absolute target address menjadi 3 (address 3 untuk awal dari Person, dan 0 untuk relative offset. $0+3=3$). Dilanjut dengan meload literal "Budi" pada address tersebut. Dan kemudian dilakukan store indirect yang melakukan pop value "Budi" dan target address 3 dan menyimpan value tersebut pada address.

3.5. Kasus 5

```
program test6;

type
  Person == record
    nama: String;
    umur: Integer;
  end;

var
  p: Person;

begin
  p.nama := 'Budi';
  p.alamat := 'Bandung';
end.
```

Output program:

```
[!] Semantic Analysis completed with errors:
Semantic Errors:
  Line 14, Column 5: [Semantic Error] at Line 14, Col 5:
Undefined record field: alamat
  Line 14, Column 5: [Semantic Error] at Line 14, Col 5: Type
mismatch in assignment
[!] Semantic analysis completed with errors.
[!] Cannot generate or run code with semantic errors.
```

Penjelasan:

Di sini alamat pada type Person sama sekali tidak didefinisikan, sehingga ditemukan error dan code generator serta interpreter tidak berjalan sama sekali.

4. Kesimpulan dan Saran

4.1. Kesimpulan

Pada Milestone 4, kami mengimplementasikan Immediate Code Generation dan sebuah Interpreter untuk mengeksekusi intermediate code. Beberapa hal yang dapat disimpulkan dari perancangan dan implementasi yang telah dilakukan adalah sebagai berikut.

Pertama, CodeGenerator menerjemahkan Decorated AST menjadi vector Instruction menggunakan helper emit/emitLiteral, mencatat panggilan yang belum terisi untuk backpatch, dan memakai SymbolTable untuk lev/adr.

Kedua, IR dirancang melalui setiap instruksi direpresentasikan oleh Instruction yang berisi opcode, level, dan operand, dan instruksi dapat membawa literal melalui RuntimeValue. RuntimeValue diimplementasikan sebagai variant sehingga instruksi LIT bisa di-assign nilai secara langsung.

Ketiga, Interpreter adalah VM stack-based dengan sp, bp, pc dan activation record [static link, dynamic link, return address]. Ia mendukung INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET; executeOpr menangani aritmatika, perbandingan, dan I/O.

Singkatnya, Milestone ini menghubungkan Decorated AST dengan Interpreter melalui Immediate Code Generation yang ringkas dan modular, menyediakan basis yang jelas untuk perluasan fitur dan pengujian lebih lanjut.

4.2. Saran

- Billie Bhaskara Wibawa - 13524024

Wak deadline nya tengah uas T-T

Memang pasti semua sedang sibuk tapi I think dari jauh udah obvious bahwa deadline tengah uas pasti bakal ke extend, and itd be nice kalo diextendnya udah dari lama.

- Raysha Erviandika Putra - 13524050



-1 deadline -1 sanity

- Muhammad Naufal Romi Annafi - 13524058



idk what's the purpose of life

- Dzaki Ahmad Al Hussainy - 13524084

Deadlinya kurang lama :D

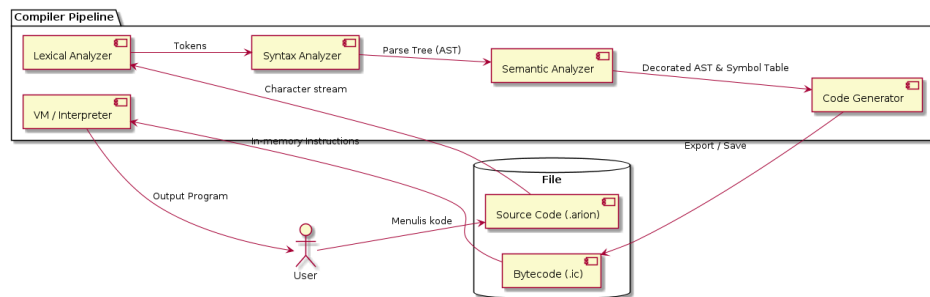
5. Lampiran

5.1. Repository

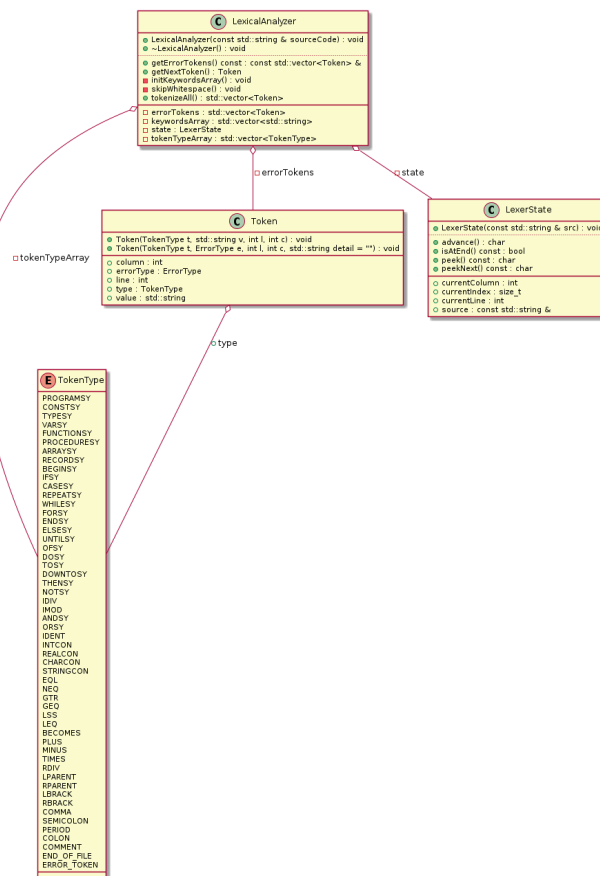
Github: [JEE-Tubes-IF2224-2026](https://github.com/JEE-Tubes-IF2224-2026)

5.2. Diagram (untuk gambar diagram yang lebih jelas dapat dicek di repository di folder doc/diagrams)

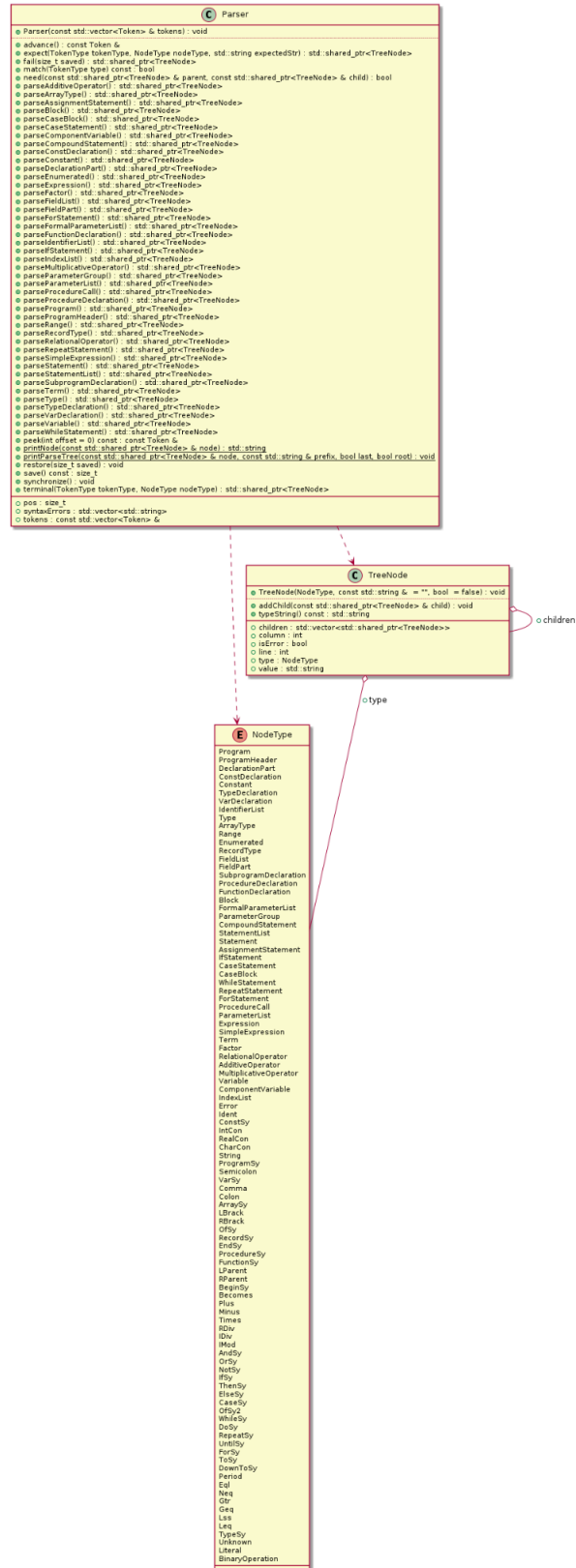
5.2.1. Compiler Pipeline



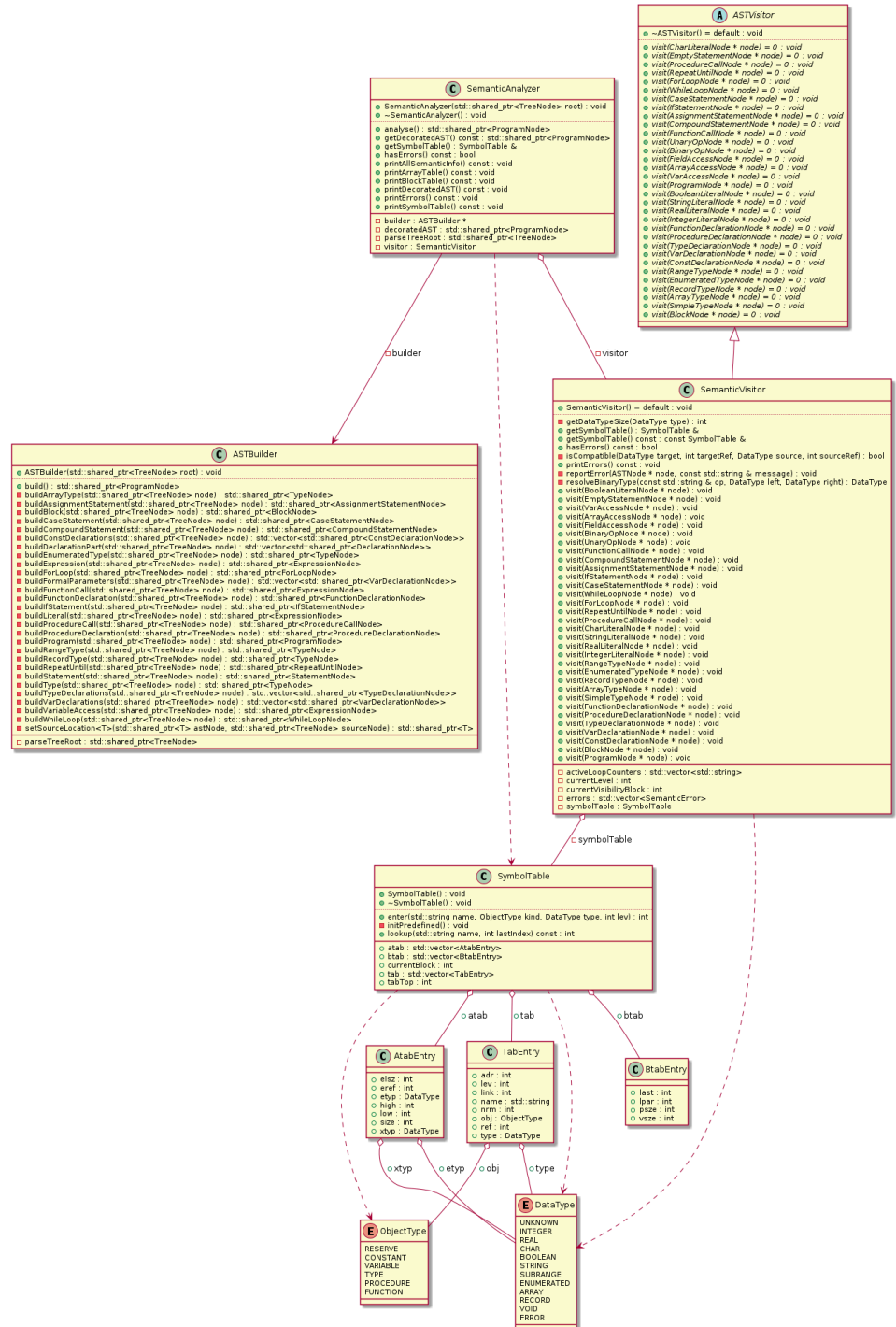
5.2.2. Lexical Analyzer



5.2.3. Syntax Analyzer



5.2.4. Semantic Analyzer



5.2.5. ASTNode

Ervandika Putra			
Muhammad Naufal Romi Annafi	13524058	Laporan, Koding	25%
Dzaki Ahmad Al Hussainy	13524084	Laporan, Koding	25%

6. Referensi

"Compilers - Principles, Techniques, and Tools (2006)"

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)

Guidebook Konvensi Intermediate Code

<https://docs.google.com/document/d/1pAy0sLZaSyITLXS1yBc7Snu4dHsAE3fJXbY24DkHeic/edit?usp=sharing>